

Aircraft Simulation – Hardening and Extension

CP2561 Project Brief (Spring 2026)

Instructor: Shivaji Patil, College of the North Atlantic

Due: 10 June 2026, 11:59 PM

Total marks
25

Group size
1 or 2 students

Due date
10 June 2026 – 11:59 PM

Project Overview

You will receive a working Java aircraft-simulation codebase as your starting point. It already contains a multithreaded Swing GUI, a physics-based attitude-control system for roll, pitch, and yaw, a live OS-level resource monitor, and a small amount of cross-platform support code. Your job is **not** to rebuild it from scratch. Your job is to extend it in three specific ways, each chosen because it forces you to read the existing code, understand it, and integrate cleanly with what is already there.

This project is worth **25 marks** in total: **15 marks for the implementation work** and **10 marks for a short live demo at submission time**. Students may work alone or in **groups of two**. Groups share a single grade by default, but if it becomes clear during the demo that only one member can defend the code, marks may be split between the partners.

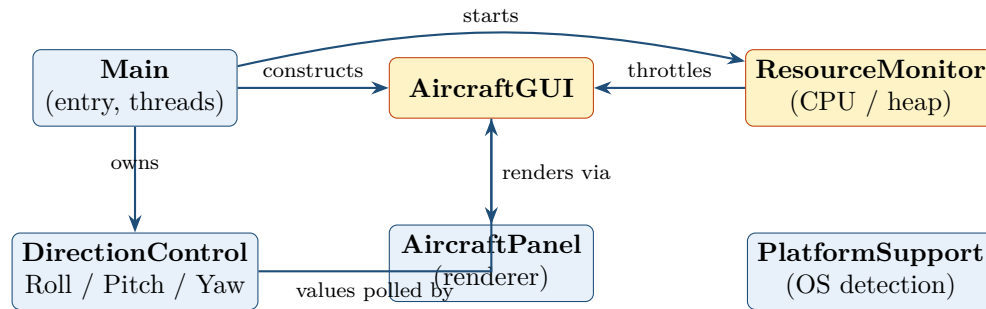
AI policy. GitHub Copilot and Copilot Chat are the only AI assistants permitted on this project. Any other large-language-model tool – including Anthropic Claude, Claude Code, ChatGPT, GPT-4 / GPT-5, Google Gemini, Cursor, Codeium, Tabnine, and any locally-hosted model – is **not** permitted. Use of any non-permitted AI tool results in **zero marks on the entire project**, with no warnings and no partial credit. The rationale is that Copilot integrates with the IDE and produces a logged, auditable trail of suggestions, which matches how professional development teams currently sanction AI assistance.

Every interaction with Copilot must be logged in a file called `PROMPTS.md`. The format is described later. An empty or fabricated log is treated as academic dishonesty and earns zero on the work portion.

You will need a JDK version 17 or later, Git for version control, and any IDE or text editor of your choice. The starter code runs on Ubuntu, macOS, and Windows; you should test on whichever operating system you are using before submission.

Starter codebase at a glance

The diagram below summarises how the starter code is wired together. Read these classes before you write your first line of new code.



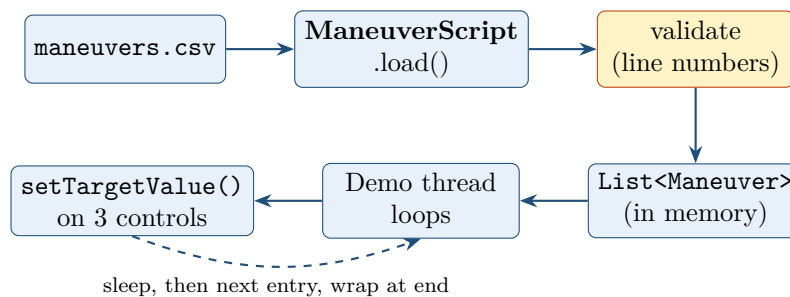
The Three Tasks (15 marks total)

You must complete all three tasks. Each is worth 5 marks. They are independent in the sense that you can attempt them in any order, but they share the same codebase, so changes you make in one task will be visible to the others. Plan accordingly.

Task 1 – Maneuver script loaded from a file (5 marks)

In the starter code, the aircraft executes a hard-coded sequence of flight maneuvers. The sequence lives inside `createAutomatedDemoThread` in `Main.java`: stable level flight for eight seconds, an ultra-gentle right turn for twelve seconds, back to level for eight seconds, an ultra-gentle left turn for twelve seconds, a gentle climb for ten seconds, a gentle descent for ten seconds, and a long stable period before the loop repeats. The target values and durations are spread across roughly sixty lines of Java code that mix configuration data with the threading logic that drives the simulation.

Your task is to extract this configuration into an external file. The simulation reads the file at startup, stores the maneuvers in memory, and executes them in order. When the script reaches its final entry, the simulation loops back to the first entry and continues indefinitely.



The file format must be CSV with a header line. The four columns are: seconds to hold the targets, target roll in degrees, target pitch in degrees, target yaw in degrees. A typical script:

```
seconds,roll,pitch,yaw
8,0,0,0
12,2,0,2
8,0,0,0
12,-2,0,-2
10,0,-5,0
```

```
10,0,3,0
10,0,0,0
```

The file path is configurable through `--script <filename>`. If no flag is provided, the simulation looks for `default_maneuvers.csv` in the current working directory. If neither file exists, print a clear error to standard error and exit with a non-zero exit code. **No stack traces, no silent fallback to hard-coded values.**

Your script reader must validate the file properly. Malformed lines, non-numeric values, and out-of-range targets (roll/yaw: $\pm 180^\circ$, pitch: $\pm 90^\circ$) must each produce a plain-English error that names the line number and the problem – for example: `Script error on line 7: expected 4 fields but found 3`, or `Script error on line 4 field 2 ("roll"): "abc" is not a number`. Blank lines and lines beginning with `#` should be treated as comments and skipped.

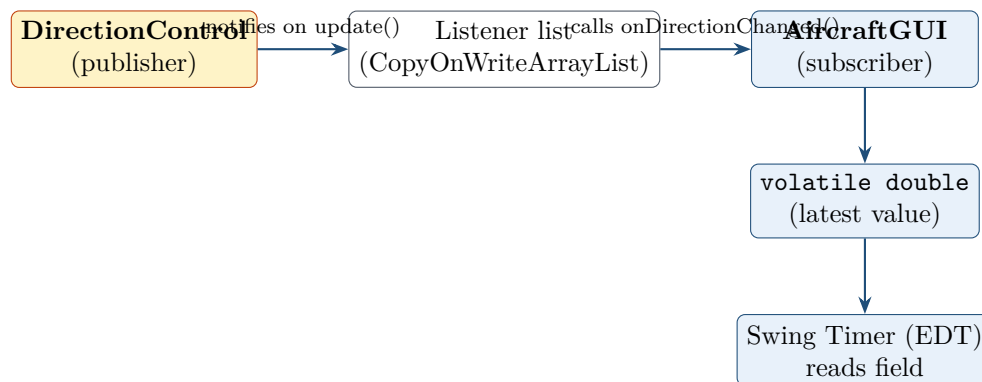
Add a new class (suggested name `ManeuverScript`) that owns loading, validation, and the in-memory representation. The demo thread in `Main.java` should shrink to a short loop that iterates through whatever `ManeuverScript` provides.

When the simulation runs with a script that contains the original maneuvers, behavior on screen must be indistinguishable from the starter code. You must submit two example script files: `default_maneuvers.csv` (reproducing the original sequence) and a second file with at least three different kinds of errors, used to demonstrate your error messages during the demo.

Task 2 – Observer pattern for direction updates (5 marks)

The starter code moves attitude data from the simulation to the GUI through **polling**. The Swing display timer wakes every 16 ms and calls `getCurrentValue()` on each `DirectionControl`, whether or not anything has changed. Polling is simple but wasteful, and the displayed value can lag the simulation by up to one frame.

Replace polling with the **Observer pattern**. The `DirectionControl` publishes a change event whenever its value moves; the GUI subscribes once and reacts to events.



Define a Java interface, for example `DirectionControlListener`, with a single method `void onDirectionChanged(DirectionControl control)`. Give `DirectionControl` two methods, `addListener(DirectionControlListener)` and `removeListener(DirectionControlListener)`, backed by a thread-safe collection (the simplest correct choice is `java.util.concurrent.CopyOnWriteArrayList`; a synchronized `ArrayList` is acceptable if you can justify the choice in your comments). Every time `update()` changes the current value, iterate through the listener list and invoke the callback on each one.

Modify `AircraftGUI` so that in its constructor it registers listeners with all three `DirectionControl`

instances, stores the most recent value in a `volatile double` field (or an `AtomicReference<Double>`) per axis, and stops calling `getCurrentValue()` from inside the Swing timer.

Pay careful attention to threading. Notifications happen on the simulation thread in `Main.java`. The GUI consumes them on the Event Dispatch Thread. Values written by the listener must be safely visible to the EDT, and your code must not call any Swing method directly from inside the listener (doing so violates Swing's threading model).

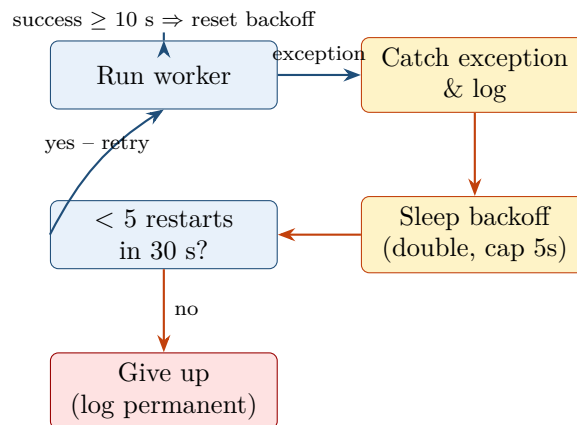
Required artefact: a short comment block, in code or in your `PROMPTS.md`, that explains in plain English how your code guarantees safe publication of the value across threads. If this comment is missing, the maximum mark for Task 2 is 3 out of 5.

When you are finished, the aircraft on screen must behave exactly as before. Only the internal mechanism has changed.

Task 3 – Self-healing worker threads (5 marks)

The simulation depends on several background threads: a turbulence thread, an automated demo thread, the resource monitor, and several internal schedulers in `AircraftGUI`. If any of these throws an uncaught exception, the thread dies silently. The JVM keeps running, but the simulation degrades in a way the user only notices indirectly.

Your task is to make the simulation **self-healing**. When a worker thread crashes, catch the exception, log it clearly with the worker's name and stack trace, wait briefly, and restart the work. Use exponential backoff: start at 100 ms, double after each failure (200, 400, 800, ...), cap at 5 seconds. After a worker runs successfully for at least 10 consecutive seconds, reset the backoff to 100 ms.



Restart attempts must be **limited**. If a single worker accumulates five restarts within 30 seconds, give up permanently and log: `worker "turbulence" exceeded restart budget; will not be restarted`. The rest of the simulation must continue.

Add a class (suggested name `SupervisedRunner`) that takes a worker name, the `Runnable` to run, and a `BooleanSupplier` that says whether the simulation is still running. Its own `run` method is a loop that invokes the work, catches `Exception` and any `RuntimeException`, performs the logging and backoff above, and iterates. The loop exits cleanly when the supplier returns false.

Apply the supervisor to **at least three** existing threads: the turbulence thread, the automated demo thread, and the resource monitor. The supervisor is not necessary for short-lived setup threads or the Swing EDT.

Test hook: add a CLI flag `-inject-failures` that makes the turbulence thread throw a `RuntimeException` at the 3-, 6-, and 9-second marks. With the flag set, the simulation must keep running for at least 60 seconds after the burst. Your log should clearly show three exceptions caught, three restarts with increasing backoff, and the simulation recovering.

Common AI trap. The first suggestion you will see is to wrap the work in `try { ... } catch (Exception e) { e.printStackTrace(); }`. That is **not enough**. After the catch returns, the `Runnable` completes and the thread exits. Your code must put the catch *inside a loop* and re-invoke the work. Submitted code that catches without retrying earns no more than 2 out of 5.

PROMPTS.md – the AI interaction log (mandatory)

Submit a file named `PROMPTS.md` in the root of your project. This is a chronological log of every interaction you had with **GitHub Copilot** or **Copilot Chat**. These are the only AI assistants permitted; any other LLM is grounds for an automatic zero on the project. Inline Copilot suggestions may be grouped by file or by session if there are many small ones; every Copilot Chat conversation must appear as its own entry. Pair-programming with a classmate is not subject to this log, but copying code from a classmate is plagiarism regardless of AI involvement.

Each entry must contain: a heading with session number and timestamp; the task you were working on; the AI tool used; the prompt **copied verbatim** (no paraphrasing); a brief summary of the suggestion; and your decision – “accepted as written”, “accepted with modifications”, or “rejected” – with one or two sentences of why, especially when you modified or rejected.

Example entry:

```
## Session 14 - 2026-06-03 14:32
Task: Task 2 (Observer pattern)
Tool: GitHub Copilot Chat
Prompt (verbatim):
> How do I make a Java class notify other objects when one of its fields
changes?
Suggestion summary:
Copilot suggested using java.util.Observer and java.util.Observable from the
standard library, with calls to setChanged() and notifyObservers().
Decision: Rejected
Why: java.util.Observer has been deprecated since Java 9 because it is not
type-safe. I wrote a custom DirectionControllListener interface instead and used
CopyOnWriteArrayList for thread-safe registration.
```

`PROMPTS.md` is graded as a **multiplier** on your work mark, not as a separate score. A strong log (specific prompts, evidence of iteration, at least one example of catching the AI being wrong) earns the full multiplier of 1.0 – meaning your task marks count in full. A vague log earns 0.6. A missing or fabricated log earns 0.0, meaning zero on the entire 15-mark work portion regardless of how well the code runs. Timestamps are cross-checked against Git history.

You are not being graded on how *much* AI you used. You are being graded on how *honestly* and *thoughtfully* you used it.

The Demo (10 marks)

At submission, every student attends a 5-minute live session with the instructor. Solo students go one-on-one; groups of two attend together, and both members must participate. The session is graded on the spot.

The demo has three parts. **Run it (2 marks)** – start the simulation with your custom maneuver script, show the resource monitor and the GUI updating, restart with `--inject-failures` and show recovery. If the demo crashes, this part earns zero. **Random code walkthrough (4 marks)** – the instructor points at a random method in your code; you explain it in plain English. **IDE and PROMPTS.md must be closed** during this part; a non-editable source viewer is allowed. **Design defense (4 marks)** – the instructor asks “why did you pick X over Y” on one design decision (lock type, listener thread, backoff parameters, file format). You name the alternative, explain the trade-off, and connect it to the constraints of this project.

For groups of two, the instructor directs different parts to different members. If one partner can run the demo but cannot defend the code, that partner’s demo mark is halved (5 instead of 10) while the other partner’s mark is unaffected. This is how uneven contribution is detected.

Rubrics

Each rubric uses three levels only – full / partial / zero – so a submission can be marked in roughly two minutes.

Task 1 – Maneuver Script from File (out of 5)

Marks	What earns this mark
5	Loads from file, validates lines with clear errors (line numbers), looping works, default script reproduces original behavior.
3	Loads and runs for valid input but error handling is weak or missing.
0	Does not work, or not attempted.

Task 2 – Observer Pattern (out of 5)

Marks	What earns this mark
5	Listener interface defined, thread-safe registration, all three axes publish, GUI stops polling, safety comment present.
3	Listeners wired but polling still present on some axis, or thread-safety comment missing.
0	Polling not replaced, or not attempted.

Task 3 – Self-Healing Threads (out of 5)

Marks	What earns this mark
5	Supervisor loop with exponential backoff and restart budget, applied to ≥ 3 threads, <code>-inject-failures</code> recovers cleanly.
3	Catches and restarts on a single failure but missing backoff, budget, or fails on bursts.
0	Thread still dies on first exception, or not attempted.

PROMPTS.md multiplier (applied to the sum of Task 1+2+3 marks)

Multiplier	Quality of log
1.0	Honest log: specific prompts, evidence of iteration, at least one case of rejecting/modifying a bad suggestion.
0.0	Missing, empty, or fabricated (contradicted by git history) – zero on the 15-mark work portion.

Demo (out of 10)

Marks	Criterion
2 / 1 / 0	Run it (out of 2): full – custom script loads and <code>-inject-failures</code> recovers; partial – runs with minor issues; zero – does not run.
4 / 2 / 0	Random code walkthrough (out of 4): full – clear line-by-line explanation; partial – gets the top-level intent but vague on specifics; zero – cannot explain the code.
4 / 2 / 0	Design defense (out of 4): full – names the alternative and explains the trade-off; partial – recognises there was a choice but cannot defend it; zero – no answer.

Submission and Academic Integrity

Submit a single ZIP archive through the course LMS, named `cp2561_project_<lastname1>_<lastname2>.zip` (one name for solo). Contents must include: all `.java` source files; `default_maneuvers.csv` and at least one intentionally-broken script for Task 1; `PROMPTS.md`; a one-page `README.md` with one sample command line for each task scenario; and a git history export (`git log --pretty=oneline > git_history.txt`). Do not submit `.class` files, IDE folders (`.idea`, `.vscode`), or large data files left over from the starter code's logging.

Late submissions lose 10% of the project's marks per calendar day, starting the moment the deadline passes. After three days the project earns zero. Contact the instructor *before* the deadline if you anticipate a problem; one extension request per project may be granted for documented reasons.

Academic integrity. The only AI assistants permitted on this project are GitHub Copilot and Copilot Chat. Use of any other LLM (Anthropic Claude, ChatGPT, Gemini, Cursor, Codeium, locally-hosted models, etc.) is an automatic zero on the project and may be referred to the program coordinator. This rule applies regardless of how small the non-permitted contribution was, and regardless of whether you disclose it after the fact.

Plagiarism, defined as submitting work that is not your own without attribution, is also grounds for a zero. Copying from another student or a public solution is plagiarism whether or not an AI assistant was involved. Two submissions that match too closely will both be investigated, and the question of who copied from whom is not the instructor's to resolve.

If you are unsure whether something counts as appropriate AI use, ask before submitting. The general rules of thumb: only Copilot is allowed; every Copilot interaction must be in `PROMPTS.md`; and you must be able to explain every line of your code from memory during the demo. If any one of these three conditions is not met, your AI use is not appropriate for this project.

Read first, type second. Most of the marks on this project are earned by reading the starter code carefully before you change anything. Spend your first session reading, not writing.